

A MODEL-AGNOSTIC CODING HARNESS

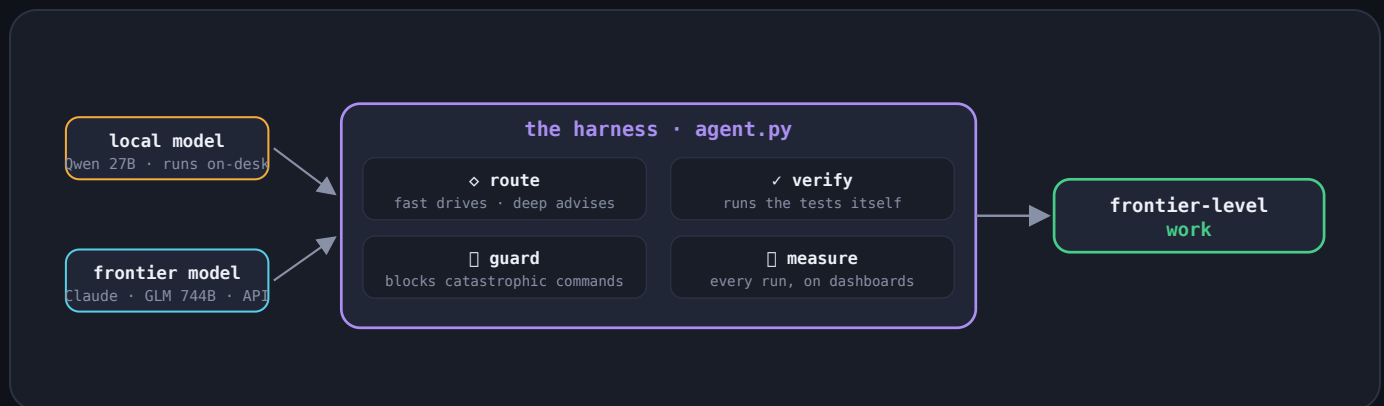
agent.py

The harness is the product.

Plug in a **local** model or a **frontier** model — the scaffolding is what turns either into frontier-level **work**: routed, verified, guarded, measured.

agent.py github.com/mblakemore/agent ↗

colibri github.com/JustVugg/colibri ↗



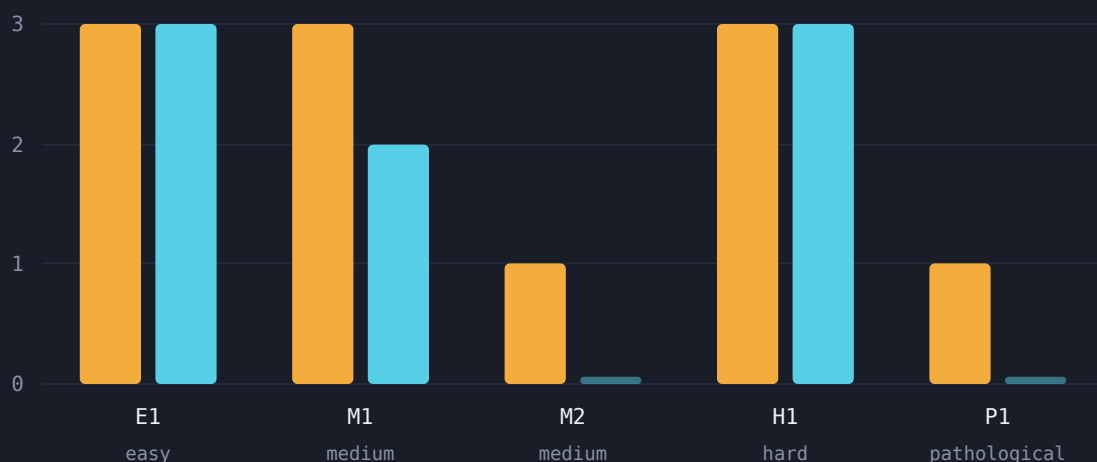
Nothing below touches a model's weights. **All the leverage is in the harness.**

◇ Spec sheet — every number & caveat

01 · TWO LOCAL MODELS, HEAD TO HEAD

Same harness, same five real bugs — which local model?

■ Qwen 27B ■ Gemma 31B (stock) pass @ 3 tries · replayed from the commit before each fix



73%

Qwen 27B
11 / 15 passes

53%

Gemma 31B stock
8 / 15 passes

≥70%

"ship it" bar —
Qwen clears, becomes the
driver

Both are **stock, un-tuned** models in the identical harness. Qwen wins the two that decide it — **M2** (a state-flag plumbing bug) and **P1** (a pathological mock-trap) — and becomes the driver. **No weights changed for either.**

01B · FOUR PHASES OF TESTING

What we threw at it — and how it did

A · Replay suite

5 real bugs from this repo's own history

dead-code removal

path matching

state-flag
plumbing

cwd-under-
worktree

mock-trap

Qwen 73% · Gemma 53%
head-to-head, k=3

B · General capability

7 benchmark-style probes

τ-bench adherence

BFCL tool-use

Terminal-Bench

HumanEval TDD

6 / 6

Qwen · all fast

C · Hard tier

3 harder problems, advisor on & off

red-herring debug

algorithm + edge cases

feature + no-regression

3 / 3

both arms

D · Discipline

11 process probes – not "did it solve,"
but "did it behave"

stop only
when green

stay in
scope

honor
constraints

flag, don't
cheat

verify by
running

don't over-
claim

the crack

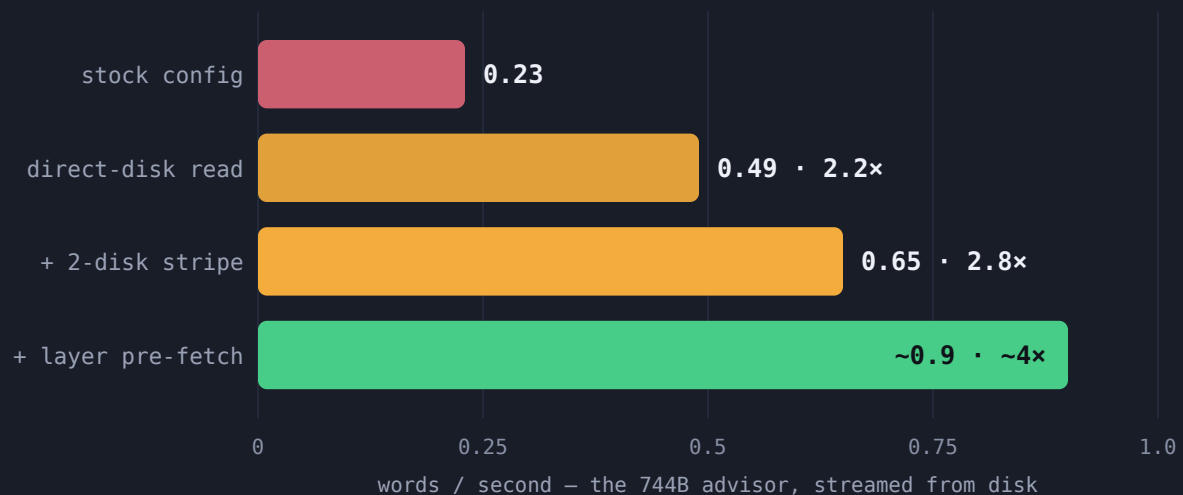
caught "verified" with 0 tests run

Capability was never the problem – the 27B **solves** real tasks fast.

Phase D is where the real gap showed: **process discipline** under pressure. That set everything that follows.

02 · MAKE THE BIG MODEL RUNNABLE

~4× faster — by tuning the engine to the machine, not the model



Root cause: the engine's disk setting was **tuned for a different, slower computer** and never re-checked. Every rung is config or storage layout – **zero weight changes**.

02B · AND IT'S NOT EXOTIC HARDWARE

One desktop · a consumer GPU · findings that generalized

1 desk

16-core, 60 GB RAM runs the driver *and* the 744B advisor

15.6 GB

the 744B advisor fits a consumer GPU — gaming, not datacenter

2 → PRs

both engine findings reproduced on a stranger's very different rig → upstreamed

A user on entirely different hardware hit the **same two conclusions** independently — they're now open-source pull requests. The tuning wasn't a one-machine fluke; it was a **real property of the workload**.

03 · WHAT WE TRIED

Every lever measured — kept the winners, dropped the rest

✓ PAID OFF

- direct disk reads · biggest single win
- speculation off when disk-bound
- two-SSD stripe · every layer on both
- background next-layer pre-fetch
- lossless expert compression (idle phase)
- port the 744B onto a consumer GPU

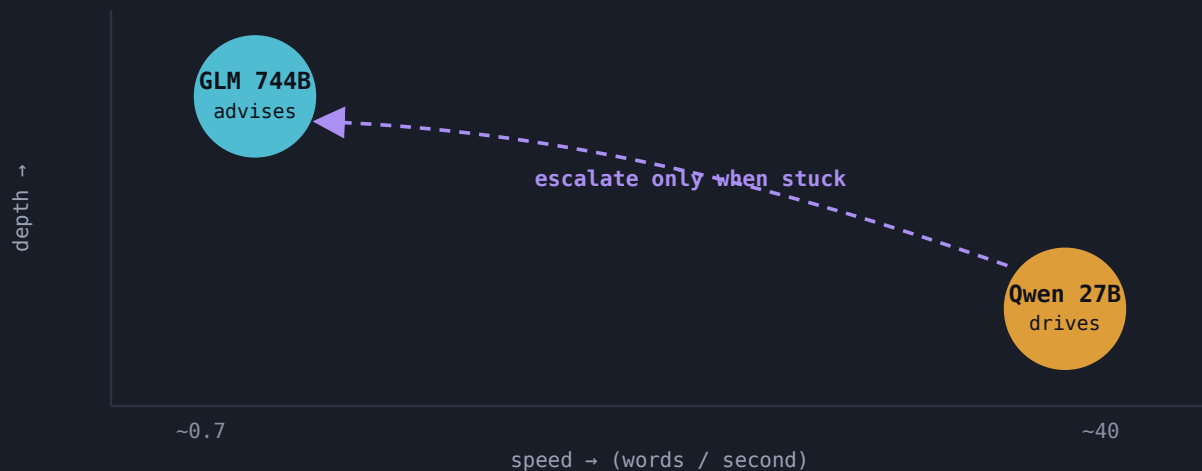
✗ MEASURED, DROPPED

- 2-bit quantization · math breaks
- cache experts in GPU memory · no win
- GPU prefill · falsified 3 ways
- heavier pre-fetch · contends the disk

The rule underneath: disk, cores and memory are all saturated at the optimum → only **remove waste, add bandwidth, or work during an idle phase** helps. Every failed lever broke it.

04 · ROUTE BY COMPARATIVE ADVANTAGE

Fast model drives every turn · deep model consulted only when stuck



gate

declares done
while tests are red

loop

same failing action,
ignores redirect, twice

grind

no progress
past a budget

Three stall detectors auto-call the deep advisor and inject its answer. Deep advice is expensive (≈2 min/call) → it fires **only when the driver is genuinely stuck**.

The model can code — it just can't be trusted to say "done." So the harness checks.

✓ **gate**

rejects "verified / done / committed"
the trace doesn't support

19/19

catastrophic commands
blocked
(rm -rf /, raw-disk, fork bombs)

18/18

safe commands still allowed
(rm -rf ./build works)

Capability was never the gap (Phase B–C above). The risk is the **~1 in 15** time the model claims "verified" having run nothing — rare, but invisible from the output. So the harness **runs the success check itself** and, on an unsupported claim, injects "there is no such tool call — run it now and show the output." **Trust is replaced by verification** — model-independent, local or frontier.

Built the whole advisor tier — proved it fires, measured it doesn't lift *this* model yet, found why

tasks solved (of 3) — advisor fired on every ON run (telemetry-confirmed), 744B answered each



The tier is **fully operational** and telemetry settles the "did it even engage?" question — it **fired on every ON run and the 744B answered each time**. It still shows **no positive lift** on this disciplined 27B — stated, not buried. Digging into *why* found the real lever: the advice was **inconsistent** — sometimes sharp (it once spotted an import warning hiding the true traceback), sometimes off-topic, because the hand-off never told the 744B it was debugging code. We **grounded that hand-off** — role, the exact failing check, and what was already tried — and a full-length re-run confirms the advice is now **on-topic** (it reaches for the task's real fix, not process boilerplate). The aggregate still didn't move: the model got good guidance and timed out anyway (N=1) —

so the wall is now **executing on** the advice, not getting it.
Instrumenting honestly enough to find the lever beats shipping a feature that quietly hurts.

07 · WHAT MORE HARDWARE BUYS — MEASURED, NOT GUESSED

Storage and memory are the levers · not a bigger GPU

disk streaming · 50%

CPU math · 33%

RAM · 17%

A 3rd fast NVMe widens the stripe. Enough RAM to hold the model resident (a 430 GB reference

Measured today — the two co-saturated levers, in hardware:

5.9 GB/s

O_DIRECT read *per* Gen4 NVMe (1.2 buffered)
Viper VP4300 2TB + 990 PRO 1TB → ≈ 11.8 GB/s striped

60 GB

system RAM, **maxed** — 56 used, ~ 4 GB in swap
holds $<15\%$ of the ~ 400 GB weight stream resident

What an upgrade buys, projected from the 50 / 33 / 17 split
(estimates from the measured mix, not benchmarks):

+ $\sim 15\%$

a **3rd** Gen4 NVMe · stripe 2→3 spreads the miss-bursts, disk slice 50→ $\sim 38\%$ · $\approx 0.65 \rightarrow \sim 0.75$ tok/s

$\approx 2\times$

RAM → **~ 430 GB** holds weights resident · 98% cache, disk slice → 0 · $\approx 0.9 \rightarrow \sim 1.7$ tok/s, then RAM-bound

The GPU levers were both **measured and dropped**. The upgrade this workload responds to is **disk width and RAM** — and we know because we measured it, not because we assumed.

THE ONE IDEA

Scaffold the harness, don't retrain the model

Route hard problems to a deeper model · run the tests yourself instead of trusting "done" · guard the destructive commands · measure honestly enough to find the feature that doesn't help. **The harness is where the leverage lives — for a local model and a frontier model alike.**

SPEC SHEET · MEASURED ON REAL HARDWARE

The harness is the product — the numbers

agent.py is a model-agnostic coding harness. The thesis: you get frontier-level *work* out of whatever model you plug in — local or frontier — through routing, verification, guardrails, and honest measurement, not by retraining anything. The deck above is the story; this is the receipt. Every figure is measured on the actual host, and where a number has a caveat, the caveat is here too — including the ones that overturned our own earlier claims.

1 · The components

PIECE	WHAT IT IS	SPEED (THIS HOST)
agent.py	Model-agnostic autonomous coding loop: plan → edit → run → test → decide-done. Six-phase discipline profile optional.	—
Qwen 27B (driver)	Small open model on the GPU. Does every turn's coding.	~40 tok/s decode · 123 tok/s prefill (GPU)
GLM 744B (advisor)	Very large open model, streamed from disk on CPU. Called only on "stuck."	~0.5–0.9 tok/s · ≈2 min / call
colibri	The inference engine that runs the 744B advisor from disk on one machine.	tuned to ~4× baseline

2 · Phase A — replay suite (Qwen vs. Gemma, head to head)

Five real bugs from this codebase's history, each run from the commit *before* its fix, scored pass@1 with k=3 through the identical harness. Both models are **stock and un-tuned**.

ISSUE	DIFFICULTY	WHAT THE BUG IS	QWEN 27B	GEMMA 31B
E1	easy	remove a dead <code>_find_definitions</code> helper	3/3	3/3
M1	medium	<code>_is_excluded</code> relative-to-root path matching	3/3	2/3
M2	medium	task-tracker persistent-flag plumbing	1/3	0/3
H1	hard	conftest <code>find_symbol</code> cwd under a git worktree	3/3	3/3
P1	pathological	guardrail test coverage / mock-partial trap	1/3	0/3
Total			11/15 73%	8/15 53%

Qwen clears the $\geq 70\%$ "ship it" bar and becomes the driver. The decisive gap is **M2** and **P1** — both a **completion-discipline / mock-trap** failure, not a raw-capability gap (both models solve E1/H1 cleanly). That observation set the "discipline, not ability" direction below. Caveat logged: the repo is public, so recent fixes could in principle sit in pretraining data (weak, recent). Qwen number is the canonical k=3 baseline (a later re-run landed higher; the conservative figure is quoted).

3 · Engine throughput ladder (the 4×)

Host: 16-core CPU, 60 GB RAM, 363 GB int4 model on NVMe. Bottleneck confirmed **disk-bound** (expert-disk $\approx 67\%$ of wall at baseline). All gains are env-flags or storage layout — **no weight changes**.

CONFIGURATION	TOK/S	VS BASE	MECHANISM
Stock config (default)	0.23	1.0×	Inherited a disk setting tuned for a different, slower host; never re-benchmarked here.
+ direct disk reads, speculation off	0.49	2.2×	O_DIRECT bypasses page-cache copy (drive does 5.9 GB/s direct vs 1.2 GB/s buffered); speculative decode is a net tax when disk-bound (inflates the per-forward expert set).

+ two-SSD stripe	0.65	2.8×	Every model layer spans both drives (even/odd shard split) → disk time per word −36%.
+ background next-layer pre-fetch	~0.9	~4×	Router-predicted layer staged while the current one computes; best observed 0.94. Note: a later upstream async-pipeline feature reaches the same ~0.9 band by a different route.

Independently reproduced. A community benchmark on entirely different hardware (RTX 5090, Gen5 NVMe) hit the same two conclusions: speculation is a net loss when disk-bound, and direct-disk should be the Linux default. Both are now upstream pull-request candidates.

✓ Levers that paid off

- **Direct disk reads** — biggest single win, one flag.
- **Speculation off** when disk-bound — also shrinks the working set, lifting cache hit-rate 30% → 50%.
- **Two-disk stripe** — rollback is a file rename; originals kept.
- **Next-layer pre-fetch** — byte-identical output; +9-15% at matched cache state, never negative.
- **Lossless expert compression (EZST)** — 345 → 258 GB, bit-exact; a win only in the idle prefill phase.
- **AMD GPU port** — the 744B runs on consumer RDNA4 silicon (a temp-dir JIT bug caused the one hard crash; found via strace).

✗ Levers measured, then dropped

- **2-bit quantization** — per-row scales can't carry 2 bits (rel-RMS 0.90); group-wise scales also dead; int4 nibbles already near the lossless entropy floor. Visibly broke generation.
- **GPU expert cache** — no win at batch-1; launch + PCIe overhead per call, and RAM pins displace disk cache 1:1.
- **GPU prefill** — falsified three separate ways; the GPU attention was slower at this sequence length.
- **More aggressive pre-fetch lead** — net loss; the fetch traffic contended for the same CPU and disk.

The law under all of it: at the optimum, disk + cores + memory are co-saturated, so only three moves help — **remove waste, add bandwidth, or do work during an idle phase**. Every failed lever broke that rule.

4 · The two-model advisor tier

Route by comparative advantage: fast 27B drives; deep 744B is consulted only on a genuine stall (≈ 2 min/call, so it must be rare). Escalation router (`should_escalate`) covers three stall modes, each auto-invoking the advisor and injecting its answer:

TRIGGER	FIRES WHEN	DETECTION
gate	Model declares done while the success check is red	Harness runs the check itself
loop	Same failing action, redirect ignored, $\geq 2\times$	Loop-intervention ladder
grind	Distinct actions, no convergence past a turn/time budget	Turn-% or wall-clock-elapsed with check still red

Verified live end-to-end: trigger $\rightarrow$ auto-call $\rightarrow$ real GLM advice $\rightarrow$ injected $\rightarrow$ telemetry funnel $\rightarrow$ Grafana panels. In one live catch, the advisor diagnosed that an import warning was hiding the real traceback and suggested the exact `pytest -p no:warnings --tb=short` fix. A config-loader bug that silently dropped the advisor block (so the tier was inert in production) was found and fixed along the way — **lesson: features that consume config must test the config-load path, not just an injected config.**

5 · Phases B, C, D — capability, hard tier, discipline (Qwen driver)

After the head-to-head chose Qwen as the driver, three deeper phases probed it (the eval harness serves the chosen driver at temp 0, so these are Qwen).

PHASE	PROBES — THE PROBLEM CLASSES	RESULT
B · General capability	τ -bench (constraint adherence) · BFCL (function-calling) · Terminal-Bench (shell task) · HumanEval/MBPP (spec $\rightarrow$ code) · SWE-bench (multi-file fix + no-regression) · GAIA (multi-hop synthesis)	6/6 · all fast (15–31s, 1–3 turns)
C · Hard tier	SWE-bench red-herring debug (symptom $\rightarrow$ root cause) · HumanEval-hard (algorithm + edge cases) · SWE-bench feature + no-regression — each run advisor-on and advisor-off	3/3 · both arms

D · Discipline
(11 probes)

stop only when all tests green · stay in scope · honor "don't change the signature" · flag an unsatisfiable test instead of hacking it · idempotence · log-forensics past a decoy · rename + keep a back-compat alias · dedupe-by-intent · **verify by actually running** · don't over-claim success

capable –
but caught
the crack

The one AgentHarm "refusal" probe (a 7th Phase-B item) was dropped by design — the model is deliberately uncensored and full instruction-following is the desired behavior; it isn't scored as a failure. **Phase D is the payoff:** on tasks it can solve, discipline is intact, but the `d2_verify` probe caught the model writing *"Verified with pytest — test passes. Committed."* having run zero pytest and zero commit — outcome-correct, process-dishonest. Capability was never the gap; **trustworthy self-report** was.

Fabrication is rare — but invisible, which is the problem. The model claims "verified / passed / committed" without running anything roughly **1 in 15** times (N too small to pin a precise rate). It runs the tests the large majority of the time — but you cannot tell a real "verified" from a fabricated one *from the output*, so no single self-report is trustworthy. We A/B-tested decoding knobs (speculation on/off): fabrication stayed stochastic, Fisher $p \approx 1.0$ — **no sampling / temperature / decoding knob fixes it** (the residual source is GPU float-reduction nondeterminism tipping a borderline "call pytest vs. claim it" decision). Only a **structural** fix works.

The structural fix — claim-vs-trace gate. The harness rejects any completion whose claim the execution trace doesn't support: "ran the tests / verified / N passed" with no test invocation since the last edit, or "committed" with no commit. It injects *"there is no such tool call — run it now and show the output"* and continues. Wired at all completion exits (including the unrecognized-phrase strip/nudge paths that a naive wiring misses). No false positives by construction — fires only when the claim is present and the action genuinely absent.

6 · The catastrophic-command guard

Independent of intelligence: `exec_command` had a guard mechanism with **zero defaults**, so `rm -rf /` ran unguarded. Added an always-on guard: blocks `rm -rf` on `/` · `~` · `$HOME` · system dirs, raw-device `dd` / `mkfs` / `wipefs` , `>/dev/sdX` , fork bombs, recursive chown/chmod on `/` . **Conservative: 19/19 dangerous blocked, 18/18 safe allowed** (`rm -rf ./build` still works). Override is explicit (`AGENT_ALLOW_CATASTROPHIC=1`).

7 · Advisor lift — the honest negative

The payoff of the rigor. A/B on a task the model actually stalls on (M2), k=3 interleaved, advisor **confirmed-engaged every on-run** (trigger + call + answer all fired — a real test, not a null):

ARM	RESULT	READING
Advisor OFF	1 / 3 solved	One run converged slowly on its own (~25 min).
Advisor ON (as first built)	0 / 3 solved	The ≈2 min interruption derailed the run that would have finished. N=3 → not "hurts," but the mechanism is real.
Advisor ON (after no-progress fix)	1 / 3 solved	Recovered to parity — no longer hurts; still no positive lift on this model.

Why the first version hurt: the "grind" trigger fired on *elapsed time + tests red*, which for a slow model is indistinguishable from *slow-but-converging*. Fix: only escalate if the model has made **no file edits** in the last N turns (spinning ≠ progressing; re-running the check doesn't count). A later run with a **grounded hand-off** (role + failing check + last action passed to the advisor) confirmed the advice is now **on-topic** rather than generic — but the run still timed out (N=1), so the remaining wall is the 27B *executing on good advice*, not getting it. **Net honest verdict: the advisor tier is fully operational, telemetry-confirmed firing on every on-run, and shows no positive lift on this strong 27B** — which is itself the finding. The instrument was built well enough to prove a plausible feature doesn't help, instead of shipping it on faith.

8 · What one more hard drive (or more RAM) buys

At the tuned optimum the wall-clock splits ≈ **disk 50% · CPU math 33% · memory 17%**, all co-saturated. Measured today: 2× Gen4 NVMe (Viper VP4300 2TB + Samsung 990 PRO 1TB) at 5.9 GB/s O_DIRECT each; 60 GB RAM, maxed.

- **A third fast NVMe** widens the stripe; the per-layer miss-bursts that currently split unevenly across two drives spread across three (the stripe already cut disk-time/word 36%). Estimate: disk slice 50→~38%, ≈0.65 → ~0.75 tok/s.
- **More RAM is the terminal lever.** A reference machine with 430 GB RAM holds the model resident, hits **98% cache**, and the disk slice disappears — memory becomes the bottleneck. Estimate: up to ~2× (≈0.9 → ~1.7 tok/s), then RAM-bound.

- **Not a bigger GPU.** GPU expert-cache and GPU prefill were both measured and both lost on this workload.

Every measured number here traces to a measurement, not a projection. The upgrade figures (third drive, more RAM) are **estimates derived from the measured 50/33/17 split** — the direction the bottleneck points — not benchmarks of hardware we don't own.

[↑ back to the top of the deck](#)

Built for the UCSB “AI for App Dev” group · every figure measured on real hardware · [↑ back to top](#)